Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Pengenalan Pemrograman Tingkat Rendah dan Lingkungan Kerja

Pertemuan 1

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 1.1

Menjelaskan perbedaan bahasa tingkat tinggi dan tingkat rendah, peran bahasa assembly, gambaran *toolchain*, serta menyiapkan lingkungan kerja pemrograman assembly (TASM/Emu8086/DOSBox).

CPMK yang didukung: CPMK01 ▪ **CPL yang didukung: CPL02**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan perbedaan mendasar bahasa tingkat tinggi dan tingkat rendah;
2. menjelaskan kedudukan bahasa assembly di antara *machine code* dan bahasa tingkat tinggi;
3. menguraikan rantai proses `.asm` → `.obj` → `.exe`;
4. memasang dan mengonfigurasi Emu8086, GUI Turbo Assembler, dan DOSBox;
5. menulis, merakit, dan menjalankan program assembly pertama.

Peta Materi

1. **Tingkat Abstraksi Bahasa Pemrograman**
   tiga tingkat bahasa; kapan tingkat rendah dipakai; perbandingan ringkas
2. **Rantai Proses Pembuatan Program Assembly**
   editing → assembling → linking → loading; assembler, linker, loader; `.COM` vs `.EXE`
3. **Lingkungan Kerja dan Struktur Program Minimal**
   Emu8086, GUI TASM, DOSBox; kerangka baku program; inisialisasi `DS`; alur eksekusi

Praktik pada pertemuan ini

Merakit dan menjalankan tiga contoh program, lalu memodifikasi salah satunya.

BAGIAN

Tingkat Abstraksi Bahasa Pemrograman

Tiga Tingkat Bahasa

Bahasa tingkat tinggi	*satu baris ⇒ banyak instruksi*
Bahasa assembly	*satu instruksi ⇒ satu instruksi mesin*
Bahasa mesin	`B8 05 00h`
Perangkat keras	*gerbang logika, bus, register*

Dari atas ke bawah: semakin dekat ke mesin. Dari bawah ke atas: semakin tinggi abstraksi. Hubungan assembly dan bahasa mesin bersifat *one-to-one* — satu instruksi assembly menghasilkan tepat satu instruksi mesin.

Operasi yang Sama pada Tiga Tingkat

Satu operasi sederhana — *memindahkan nilai 5 ke sebuah tempat* — ditulis berbeda pada setiap tingkat:

Bahasa mesin

```
B8 05 00
```

deretan bit, tanpa nama simbolik

Bahasa assembly

```
MOV AX, 5
```

satu *mnemonic* untuk satu instruksi

Bahasa tingkat tinggi

```
x = 5;
```

menyembunyikan register dan memori

Inti perbedaannya

Tingkat abstraksi adalah seberapa jauh bahasa menyembunyikan detail perangkat keras dari programmer.

Kapan Pemrograman Tingkat Rendah Digunakan

- **Kendali penuh atas perangkat keras.** *Device driver*, *bootloader*, dan *firmware* memerlukan akses langsung ke register dan port I/O.
- **Kinerja dan ukuran kode.** Pada sistem tertanam dengan memori terbatas, program assembly dapat jauh lebih kecil dan lebih cepat.
- **Analisis dan keamanan.** Membaca hasil *disassembly*, menganalisis *malware*, dan menguji keamanan perangkat lunak.
- **Pendidikan.** Memperlihatkan secara eksplisit bagaimana prosesor mengambil instruksi, memindahkan data, dan mengambil keputusan berdasarkan *flag*.

Yang perlu diingat

Tingkat rendah tidak berarti "lebih baik". Keduanya alat dengan tujuan berbeda.

Perbandingan Ringkas

Aspek	Bahasa Tingkat Tinggi	Bahasa Tingkat Rendah
Kedekatan ke mesin	Jauh; mesin disembunyikan	Sangat dekat; instruksi mesin tampak langsung
Portabilitas	Tinggi, satu kode untuk banyak mesin	Rendah, terikat satu keluarga prosesor
Produktivitas	Tinggi, sedikit kode untuk banyak pekerjaan	Rendah, banyak kode untuk pekerjaan sederhana
Kendali perangkat keras	Terbatas	Penuh (register, memori, port I/O)
Ukuran hasil	Lebih besar (ada kode pustaka)	Sangat kecil, dapat dioptimalkan
Pemeliharaan	Mudah dibaca dan diubah	Sulit dibaca, rawan kesalahan kecil

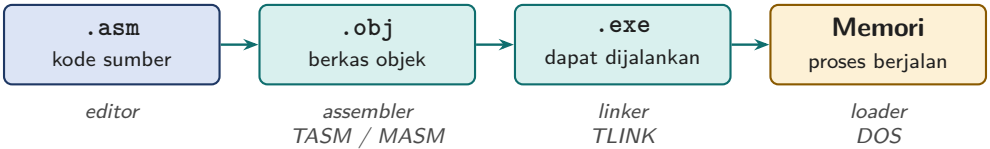
Rangkuman Bagian 1

- Tingkat abstraksi ditentukan oleh seberapa jauh sebuah bahasa menyembunyikan detail perangkat keras. Urutannya dari bawah ke atas: perangkat keras, bahasa mesin, assembly, bahasa tingkat tinggi.
- Operasi yang sama dapat dituliskan pada setiap tingkat: `B8 05 00h` (mesin), `MOV AX, 5` (assembly), dan `x = 5;` (tingkat tinggi).
- Hubungan assembly dan bahasa mesin bersifat *one-to-one*, sedangkan satu baris bahasa tingkat tinggi dapat mewakili banyak instruksi mesin.
- Semakin rendah tingkatnya, semakin penuh kendali atas register, memori, dan port I/O, tetapi portabilitas, produktivitas, dan kemudahan pemeliharaan semakin rendah.
- Pemrograman tingkat rendah dipilih ketika kendali perangkat keras, ukuran atau kecepatan kode, atau analisis keamanan menjadi pertimbangan utama.

BAGIAN

Rantai Proses Pembuatan Program Assembly

Tahapan Rantai Proses



1. **Penyuntingan** — menulis kode `.asm`.
2. **Perakitan** — assembler menerjemahkan *mnemonic* menjadi kode mesin → `.obj`.
3. **Pengaitan** — linker menyelesaikan alamat dan menggabungkan pustaka → `.exe`.
4. **Pemuatan** — *loader* memuat ke memori dan menyerahkan kendali ke instruksi pertama.

Assembler, Linker, dan Loader

Assembler Menerjemahkan kode sumber assembly menjadi kode mesin. Karena setiap instruksi assembly bersesuaian dengan satu instruksi mesin, penerjemahan ini berjalan *one-to-one* ditambah penanganan direktif dan simbol.

Linker Menyelesaikan rujukan antarberkas. Bila program memanggil prosedur di berkas lain atau di pustaka, linker-lah yang menyambungkannya dan menetapkan alamat akhir.

Loader Bagian sistem operasi yang memuat `.exe` ke memori, membuat *Program Segment Prefix* (PSP), menetapkan register segmen, dan melompat ke titik masuk.

Pembagian tugas

Assembler → kode mesin per instruksi ▪ Linker → alamat akhir ▪ Loader → memuat ke memori.

Program .COM dan .EXE

Aspek	Program .COM	Program .EXE
Ukuran maksimum	64 KB dikurangi 256 byte	Tidak dibatasi satu segmen
Struktur segmen	Satu segmen: kode, data, stack menyatu	Terpisah: segmen kode, data, stack sendiri
Titik awal	Wajib `ORG 100h`	Dinyatakan oleh `END` pada label awal
Inisialisasi DS	Otomatis sudah benar	Harus sendiri: `MOV AX, @DATA` lalu `MOV DS, AX`
Cara keluar	`INT 20h`	`MOV AH, 4Ch` lalu `INT 21h`

Jebakan Paling Sering: Register DS

Kesalahan klasik pada program .EXE

Lupa menginisialisasi register DS. Akibatnya instruksi yang mengakses variabel membaca alamat yang salah dan program mencetak karakter sampah.

Mengapa .COM tidak perlu?

DOS sudah menetapkan seluruh register segmen ke segmen program, sehingga variabel dapat langsung dialamatkan.

Dua instruksi wajib pada awal program .EXE:

```
MOV AX, @DATA  ▪  MOV DS, AX
```

Pesan Kesalahan Assembler yang Sering Muncul

Pesan	Penyebab umum	Cara memperbaiki
`Undefined symbol`	Label atau variabel belum didefinisikan	Periksa penulisan nama; pastikan label sudah didefinisikan
`Syntax error`	Salah tulis mnemonic atau operand	Periksa ukuran operand dan tanda koma pemisah
`Operand types do not match`	Ukuran operand tidak sama	Samakan ukuran, atau pakai `BYTE PTR` / `WORD PTR`
`Improper operand type`	Operasi memori-ke-memori yang tidak diizinkan	Pindahkan salah satu operand ke register dulu

Rangkuman Bagian 2

- Empat tahap rantai proses: penyuntingan `.asm`, perakitan `.obj`, pengaitan `.exe`, lalu pemuatan ke memori oleh *loader*.
- Pembagian tugas: assembler menerjemahkan *mnemonic* menjadi kode mesin, linker menyelesaikan rujukan antarberkas dan menetapkan alamat akhir, *loader* membuat PSP dan melompat ke titik masuk.
- `.COM`: satu segmen, maksimum 64 KB dikurangi 256 byte, wajib `ORG 100h`, keluar dengan `INT 20h`, dan `DS` sudah benar dengan sendirinya.
- `.EXE`: segmen kode, data, dan stack terpisah, keluar dengan `MOV AH, 4Ch` + `INT 21h`, dan `DS` wajib diinisialisasi lebih dahulu.
- Empat pesan kesalahan assembler yang sering muncul: `Undefined symbol`, `Syntax error`, `Operand types do not match`, dan `Improper operand type`.

BAGIAN

Lingkungan Kerja dan Struktur Program Minimal

Pilihan Lingkungan Kerja

Perangkat	Kelebihan	Keterbatasan
Emu8086	Emulator dengan tampilan register, memori, dan *flag* langsung; cocok untuk pemula	Menjalankan emulasi, bukan perangkat keras sungguhan
GUI Turbo Assembler	Antarmuka grafis untuk TASM/TLINK dengan *debugger* terpadu; mendekati alur kerja nyata	Perlu pemahaman struktur segmen dan linker
DOSBox	Menyediakan lingkungan DOS asli; dapat menjalankan DEBUG, TASM, TLINK, dan `.exe` lama	Antarmuka baris perintah, tanpa tampilan register otomatis

Ketiganya bebas biaya dan dapat dipasang berdampingan pada satu komputer.

Penyiapan Lingkungan Kerja

Kerjakan sebelum pertemuan berikutnya:

1. Pasang **Emu8086**, buat satu berkas program sederhana, lalu jalankan.
2. Pasang **GUI Turbo Assembler** beserta TASM dan TLINK, lalu rakit ulang program yang sama.
3. Pasang **DOSBox**, tambahkan perintah `mount` ke direktori kerja, dan jalankan `DEBUG` di dalamnya.
4. Dokumentasikan seluruh langkah pemasangan dalam jurnal praktikum.

Target

Pada akhir pertemuan ini setiap mahasiswa sudah dapat merakit dan menjalankan satu program assembly di komputernya sendiri.

Struktur Program Assembly Minimal

Kerangka baku program assembly 8086 untuk lingkungan DOS:

```
.MODEL SMALL          ; tentukan model memori
.STACK 100h           ; sediakan ruang stack

.DATA                 ; ------- segmen data -------
    ; deklarasi variabel dan konstanta di sini

.CODE                 ; ------- segmen kode -------
START:                ; label titik masuk program
    MOV AX, @DATA     ; inisialisasi register DS
    MOV DS, AX

    ; instruksi program di sini

    MOV AH, 4Ch       ; keluar dari program
    MOV AL, 00h
    INT 21h

END START             ; akhiri dan tunjuk titik masuk
```

Peran Setiap Bagian Kerangka

Direktif / label	Peran
`.MODEL SMALL`	Model memori: satu segmen kode + satu segmen data, masing-masing maks. 64 KB
`.STACK 100h`	Menyediakan `100h` byte (256 byte) ruang stack
`.DATA`	Awal segmen data; variabel dideklarasikan dengan `DB`, `DW`, atau `DD`
`.CODE`	Awal segmen kode; seluruh instruksi yang dieksekusi
`START:`	Label instruksi pertama; harus sama dengan nama setelah `END`
`END START`	Menutup berkas dan menunjuk titik masuk eksekusi

Inisialisasi Register DS

Program .EXE tidak otomatis memiliki DS yang benar. Dua instruksi berikut **wajib** ada di awal program:

```
MOV AX, @DATA        ; salin alamat segmen data ke AX
MOV DS, AX           ; lalu pindahkan ke DS
```

Mengapa harus lewat AX?

Arsitektur 8086 melarang pemindahan nilai segera (*immediate*) langsung ke register segmen. Pemindahan ke register segmen hanya boleh berasal dari register umum.

@DATA adalah simbol bawaan hasil .MODEL, berisi nomor segmen tempat .DATA berada.

Alur Eksekusi Program Pertama

1. DOS memuat `.exe`, membuat *Program Segment Prefix*, dan menetapkan `CS` serta `IP` ke titik masuk.
2. Instruksi pertama menginisialisasi `DS` agar penunjukan variabel pada segmen data benar.
3. Instruksi program dijalankan satu per satu sesuai urutan.
4. Layanan DOS `INT 21h` fungsi `09h` mencetak string ke layar.
5. Layanan DOS `INT 21h` fungsi `4Ch` mengakhiri program dan mengembalikan kendali ke DOS.

Pola yang akan dipakai berulang

Siapkan argumen pada register → panggil layanan `INT 21h` → akhiri program.

Rangkuman Bagian 3

- Tiga lingkungan kerja yang dapat dipasang berdampingan: Emu8086 (emulator dengan tampilan register, memori, dan *flag*), GUI Turbo Assembler (TASM/TLINK bergrafis), dan DOSBox (lingkungan DOS asli dengan `DEBUG`).
- Kerangka minimal program: `.MODEL SMALL`, `.STACK 100h`, `.DATA`, `.CODE`, label titik masuk `START:`, dan `END START`.
- `.MODEL` menentukan model memori, `.STACK` menyediakan 256 byte ruang stack, dan nama setelah `END` harus sama dengan label titik masuk.
- Pada `.EXE`, inisialisasi `DS` harus melalui `AX` (`MOV AX, @DATA` lalu `MOV DS, AX`) karena nilai segera tidak boleh langsung masuk ke register segmen.
- Alur eksekusi: DOS memuat `.exe` dan membuat PSP, `DS` diinisialisasi, instruksi dijalankan berurutan, lalu `INT 21h` fungsi `09h` mencetak dan fungsi `4Ch` mengakhiri program.

Contoh 1: Mencetak Teks dengan Layanan DOS

```
.MODEL SMALL
.STACK 100h
.DATA
    pesan DB 'Halo, dunia!', 0Dh, 0Ah, '$'
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AH, 09h
    LEA DX, pesan
    INT 21h
    MOV AH, 4Ch
    MOV AL, 00h
    INT 21h
END START
```

Keluaran

```
Halo, dunia!
```

`AH` = nomor fungsi layanan DOS.

`DX` = alamat string.

String diakhiri `'$'`.

`code/asm/hello.asm`

Contoh 2: Program .COM dengan ORG 100h

Bentuk lebih ringkas: seluruh kode, data, dan stack dalam satu segmen sehingga DS tidak perlu diinisialisasi.

```
.MODEL TINY
.CODE
ORG 100h            ; program .COM dimulai pada offset 100h

START:
    MOV AH, 09h     ; cetak string
    MOV DX, OFFSET pesan
    INT 21h

    MOV AH, 4Ch     ; keluar
    MOV AL, 0
    INT 21h

pesan DB 'Selamat datang di assembly!', 0Dh, 0Ah, '$'
END START
```

Perhatikan: tidak ada `MOV AX, @DATA` maupun `MOV DS, AX`.

Contoh 3: Menampilkan Karakter Satu per Satu

Layanan DOS fungsi `02h` mencetak satu karakter pada register `DL`.

```
.MODEL SMALL
.STACK 100h
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV AH, 02h      ; cetak karakter pada DL
    MOV DL, 'A'
    INT 21h
    MOV DL, 'B'
    INT 21h
    MOV DL, 'C'
    INT 21h
    MOV AH, 4Ch
    MOV AL, 0
    INT 21h
END START
```

Keluaran

```
ABC
```

Ketiga contoh memakai pola yang sama: siapkan argumen pada register, panggil `INT 21h`, lalu akhiri program.

Praktikum: Program Assembly Pertama

Judul praktikum

Pemasangan lingkungan kerja dan menjalankan program assembly pertama ▪
Sub-CPMK: 1.1 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

Alat dan bahan

- Komputer bersistem operasi Windows
- Emu8086
- GUI Turbo Assembler + TASM + TLINK
- DOSBox
- Berkas contoh `hello.asm`

Langkah kerja

1. Pasang Emu8086, rakit dan jalankan `hello.asm`
2. Amati register `AX`, `DX`, `DS` dengan *Single Step*
3. Rakit dengan TASM + TLINK di DOSBox
4. Ubah isi `pesan` menjadi nama Anda, rakit ulang

Dikumpulkan: `hello.asm` versi modifikasi, tangkapan layar keluaran (Emu8086 dan DOSBox), dan tabel pengamatan register.

Latihan

1. Jelaskan dengan kalimat Anda sendiri perbedaan bahasa mesin, bahasa assembly, dan bahasa tingkat tinggi. Sertakan satu contoh operasi yang sama pada ketiga tingkat.
2. Mengapa hubungan assembly dan bahasa mesin disebut *one-to-one*, sedangkan bahasa tingkat tinggi tidak?
3. Sebutkan tahapan yang dilalui `program.asm` sampai dapat dijalankan. Apa yang dikerjakan assembler dan apa yang dikerjakan linker?
4. Sebutkan tiga perbedaan penting program `.COM` dan `.EXE`. Mengapa `.COM` tidak memerlukan `MOV AX, @DATA`?
5. Tulislah program yang mencetak inisial nama Anda memakai layanan `INT 21h` fungsi `02h`.

Rangkuman

- Bahasa pemrograman dibedakan menurut tingkat abstraksi: bahasa mesin, assembly, dan tingkat tinggi. Assembly berada tepat di atas bahasa mesin dan bersifat *one-to-one*.
- Pemrograman tingkat rendah dipakai untuk kendali penuh perangkat keras, ukuran/kecepatan kode, serta analisis dan keamanan.
- Rantai proses: `.asm` → assembler → `.obj` → linker → `.exe` → *loader* → memori.
- Struktur minimal program memuat `.MODEL`, `.STACK`, `.DATA`, `.CODE`, label titik masuk, dan `END`.
- `.EXE` wajib menginisialisasi `DS`; `.COM` tidak.
- Pola dasar: siapkan argumen pada register, panggil `INT 21h`, akhiri dengan fungsi `4Ch`.

Cek Pemahaman

1. Hubungan satu instruksi assembly dengan instruksi mesin yang dihasilkannya adalah ...

   1.1 satu assembly menghasilkan banyak instruksi mesin
   1.2 satu assembly menghasilkan tepat satu instruksi mesin
   1.3 banyak assembly menghasilkan satu instruksi mesin
   1.4 tidak ada hubungan tetap

2. Berkas keluaran yang dihasilkan langsung oleh assembler TASM adalah ...

   2.1 `.asm`
   2.2 `.obj`
   2.3 `.exe`
   2.4 `.lst`

3. Fungsi layanan DOS untuk mengakhiri program adalah ...

   3.1 `09h`
   3.2 `02h`
   3.3 `4Ch`
   3.4 `01h`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan praktikum: `hello.asm` versi modifikasi dan tabel pengamatan register.
- Pastikan ketiga perangkat (Emu8086, GUI TASM, DOSBox) sudah terpasang dan dapat dijalankan.
- Baca **Bab IV** — Arsitektur Mikroprosesor 8086.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil